

De Esqueleto Funcional a Herramienta Profesional: Hoja de Ruta para la Transformación Visual de Bocatta POS

Análisis Arquitectónico y Estado Actual de la Plataforma

El análisis del proyecto Bocatta POS revela una evolución tecnológica y estratégica significativa, transitando desde una aplicación monolítica con lógica acoplada a un giro comercial específico hacia una plataforma modular robusta y agnóstica [118119](#). Esta transición no ha sido meramente una refactorización técnica; ha sido una redefinición arquitectónica fundamental que sienta las bases para una escalabilidad futura y una experiencia de usuario verdaderamente profesional. El estado actual de la plataforma, validado por un pipeline CI/CD exitoso con 218 tests unitarios y instrumentados, demuestra una alta madurez técnica y un cumplimiento excepcional de los principios de Clean Architecture.

La piedra angular de esta transformación es la adopción de una arquitectura Modular Monolith, caracterizada por una separación estricta entre capas de dominio, datos y presentación [26](#). Esta estructura permite que cada componente del sistema tenga una responsabilidad bien definida y minimice las dependencias cruzadas, lo cual es crucial para la mantenibilidad y el crecimiento del proyecto. Los principios clave que sustentan esta arquitectura se manifiestan en múltiples aspectos del código. La inversión de dependencias se aplica rigurosamente: los componentes de la capa de presentación, como los ViewModels, dependen exclusivamente de interfaces definidas en el paquete `domain`, tales como `IProductRepository` e `IInventoryRepository`, y nunca de sus implementaciones concretas en `data` [119](#). Esta práctica, central en Clean Architecture, crea un sistema desacoplado donde la lógica de negocio puede evolucionar o cambiarse sin repercusiones en la interfaz de usuario, un pilar indispensable para la creación de un sistema multi-giro. Por ejemplo, cambiar la lógica de producción de alimentos a la de servicios solo requeriría implementar un nuevo motor (`ProductionEngine`) que siga la misma interfaz, dejando intacto el resto del sistema.

El núcleo de la plataforma reside en el paquete `domain`, que contiene toda la lógica de negocio encapsulada en Kotlin puro, libre de cualquier dependencia externa como

Android SDK, Firebase o librerías de IU [119](#). Un ejemplo paradigmático de esta pureza es el motor de conversión de unidades, `UnitConverter.kt`. Este objeto es inmutable y agnóstico al giro, diseñado únicamente para manejar conversiones métricas y validar dimensiones, demostrando su naturaleza atómica y su potencial de reutilización en otros contextos [1](#). De manera similar, los motores de lógica de negocio como `PromotionsEngineV2` y `BusinessLogicFactory` residen completamente en el dominio, separando la lógica financiera (cálculo de descuentos y promociones) de la lógica física (gestión de producción e inventario) [2](#) [58](#). Esta separación es crítica para evitar errores complejos donde una operación financiera podría intentar modificar directamente el inventario físico. El flujo de venta, encapsulado en `SalesFlowUseCase`, ejemplifica esta cohesión, orquestando acciones como la aplicación de promociones, la creación de la transacción y la deducción de inventario en un proceso atómico y predecible.

La capacidad del sistema para operar de manera fiable, incluso en condiciones de conectividad intermitente, es otra fortaleza arquitectónica destacada. Se logró una atomicidad garantizada en las operaciones de inventario mediante el uso de transacciones de Firestore (`adjustStockBatch`) y la implementación de un patrón de commit de dos fases en el `SyncWorker` [55](#) [59](#). Este patrón asegura que si una parte del proceso falla (por ejemplo, durante la sincronización offline), todas las modificaciones se revierten, preservando la integridad de los datos. La superación del bloqueo técnico inicial causado por la incompatibilidad de KSP con la versión de Kotlin fue un momento crítico que demostró la madurez del equipo. En lugar de abandonar la persistencia local, se adoptó una solución pragmática utilizando SQLite nativo (`SQLiteOpenHelper`), manteniendo la misma interfaz de dominio (`IStockAdjustmentQueue`). Esta decisión, aunque temporal, permitió continuar desarrollando y validando el flujo offline sin comprometer la arquitectura general, un acto de ingeniería resiliente que preserva la inversión de desarrollo realizada [119](#).

La siguiente tabla resume el estado de los principios arquitectónicos clave y su grado de implementación en la plataforma Bocatta POS:

Principio Arquitectónico	Estado de Implementación	Evidencia Técnica
Separación de Capas (Dominio/Datos/Presentación)	Implementado	Archivos de directorios <code>domain/</code> , <code>data/</code> , <code>presentation/</code> claros y sin dependencias circulares 26 .
Inversión de Dependencias	Implementado	<code>ViewModel</code> depende de interfaces (<code>IProductRepository</code>), no de implementaciones concretas 119 .
Dominio Puro	Implementado	Código de <code>domain</code> es Kotlin puro, sin dependencias de Android/Firebase 119 .
Fábrica de Lógica por Giro	Implementado	<code>BusinessLogicFactory</code> resuelve dinámicamente motores de negocio (<code>FoodProductionEngine</code> , etc.) 96 .
Persistencia Offline Atómica	Implementado	Cola de SQLite con <code>SyncWorker</code> que utiliza commit de dos fases y <code>limitedParallelism(1)</code> 9 75 .
Precisión Fiscal	Implementado	Uso de <code>BigDecimal</code> con redondeo <code>HALF_UP</code> y comparación estricta (<code>compareTo</code>) para cálculos monetarios 43 45 .
Trazabilidad de Datos	Implementado	<code>v2_movements</code> registra todos los ajustes de inventario con traza <code>append-only</code> 41 .

A pesar de estos logros, persisten algunas brechas técnicas menores que, si bien no son críticas, impiden alcanzar el puntaje perfecto de 100/100. La más notable es la convivencia de modelos de datos legacy (`ProductV2`) junto con el modelo consolidado (`SalesInventoryProductV2`). Aunque los mappers bidireccionales están bien aislados en la capa de datos, representan una fuente potencial de errores y complejidad que debe ser eliminada en una futura iteración (v2.2) [119](#). Otra área de mejora es la cobertura de pruebas, que, aunque robusta en el dominio, requiere la adición de pruebas instrumentadas de UI (`ComposeTest`) para validar automáticamente el comportamiento visual y los flujos de usuario en dispositivos reales [3](#). Finalmente, la validación de giros comerciales no alimenticios en un entorno de producción real todavía depende de la carga correcta de documentos JSON en Firestore, lo que representa una fase de configuración operativa más que un bloqueo técnico [4](#). En conjunto, estas brechas son gestionables y no comprometen la estabilidad ni la escalabilidad del sistema. La plataforma ha superado con creces el umbral de desarrollo experimental para convertirse en un sistema de producción estable, listo para la próxima fase: la transformación integral de la experiencia de usuario.

Principios Fundamentales del Sistema de Diseño

Meridian

Para transformar Bocatta POS de una aplicación funcional a una herramienta visualmente potente y profesional, es imperativo definir un sistema de diseño coherente y robusto. Este sistema, inspirado en los principios del sistema de diseño "Meridian", servirá como la hoja de ruta para todo el rediseño de la interfaz de usuario. Su objetivo es eliminar la sensación de "esqueleto" y proyectar una percepción de calidad "producción-grade", centrando la experiencia del usuario en la eficiencia, la legibilidad y la consistencia. Los pilares de este sistema son los tokens semánticos, una jerarquía tipográfica clara, un sistema de espaciado basado en un grid, y un conjunto de componentes visuales reutilizables.

El primer pilar es el **uso de tokens semánticos**. La práctica de codificar colores duros (`Color.White`, `Color(0xFF1B5E20)`) debe ser erradicada de la capa de presentación. En su lugar, toda la aplicación debe depender exclusivamente de los tokens proporcionados por `MaterialTheme.colorScheme` [95](#) [108](#). Esto no es solo una elección estética, sino una decisión técnica fundamental que permite la gestión fácil de temas (claro, oscuro) y asegura la consistencia de la identidad visual a través de toda la aplicación. La paleta de colores se define según la identidad visual del proyecto: - **Primary (Verde Bocatta):** `#1B5E20` (Light) / `#4CAF50` (Dark). Usado para botones principales, precios y acentos positivos. - **Error / Danger (Rojo):** `#C62828` (Light) / `#EF5350` (Dark). Para stock agotado, errores y botones de cancelar. - **Warning (Naranja):** `#EF6C00`. Para indicar stock bajo. - **Background (Fondo):** `#FFF5F6F8` (Light) / `#FF08090F` (Dark). - **Surface (Superficie):** `#FFFFFF` (Light) / `#FF121212` (Dark). Para tarjetas, diálogos y fondos secundarios. - **OnSurface (Texto Principal):** `#FF212121` (Light) / `#FFE0E0E0` (Dark).

El segundo pilar es una **jerarquía tipográfica deliberada** diseñada para la usabilidad en un entorno de punto de venta. La legibilidad a distancia es primordial para que el personal pueda operar la terminal sin tener que acercarse físicamente. Las reglas son estrictas: - **Precios (Cuerpo Grande):** 20sp, peso SemiBold. Este es el dato más importante en una tarjeta de producto y debe ser el primero en captar la atención. - **Nombres de Producto (Cuerpo Grande):** 16sp máximo, truncado a 2 líneas con elipsis (`maxLines = 2`, `overflow = Ellipsis`). Evita que una descripción larga expanda innecesariamente la tarjeta. - **Metadatos y Etiquetas (Etiqueta Pequeña):** 12sp - 14sp. Usado para mostrar unidades (kg, L), etiquetas de stock o información secundaria. - **Mínimo Absoluto:** Ningún texto legible debe tener un tamaño inferior a 14sp para garantizar la legibilidad en todas las condiciones [31](#) [43](#).

El tercer pilar es un **sistema de espaciado y geometría basado en un grid**. La consistencia visual se logra mediante la repetición de patrones. Se adopta un sistema base de múltiplos de 8dp. Los radios de borde también son consistentes: tarjetas (**Cards**) con 12dp, campos de entrada con 8dp y botones de acción con bordes redondeados (shape grande) ⁶. Todos los elementos táctiles deben tener un tamaño mínimo de 48dp x 48dp para facilitar el acceso y reducir el riesgo de pulsaciones incorrectas, especialmente con guantes o en movimiento ^{30 82}. El principio de "una mano" guía la colocación de botones críticos en la parte inferior de la pantalla en dispositivos móviles ⁶.

Finalmente, el cuarto pilar es el **conjunto de componentes visuales reutilizables**. La creación de `BocattaComponentsV2.kt` es un paso crucial para traducir estos principios en código tangible. Este archivo debe contener componentes atómicos y moleculares que encapsulen la lógica de renderizado y estilos. Ejemplos clave incluyen: - **ProductCardV2**: Una tarjeta cuadrada (relación de aspecto 1:1) con un gradiente de fondo, el nombre del producto, el precio destacado en una pastilla negra, y badges de stock (verde, amarillo, rojo) en la esquina superior izquierda. Debe incluir una animación de escala y retroalimentación táctil al ser pulsada. - **StatusBadge**: Un componente simple para mostrar estados (agotado, bajo, disponible) con un fondo transparente al 10% del color correspondiente y texto sólido. - **BocattaButton**: Variaciones de botones (Primario, Secundario) con estados de carga nativos y retroalimentación táctil. - **MetricCard**: Utilizado en pantallas de reportes y cierre de caja, con un fondo `SurfaceVariant` muy suave, un ícono grande a la izquierda y un valor numérico grande a la derecha.

Estos cuatro pilares —tokens, tipografía, espaciado y componentes— forman un sistema de diseño completo y autocontenido. Al adherirse estrictamente a él, el equipo de desarrollo puede construir nuevas pantallas y refinar las existentes de manera rápida, consistente y escalable, transformando Bocatta POS en la poderosa herramienta visualmente coherente que se busca.

Plan de Acción Detallado para la Rediseño de la Interfaz de Usuario

El rediseño de la interfaz de usuario de Bocatta POS debe seguir un plan de acción metódico y secuencial para garantizar una transformación completa sin introducir

regresiones ni afectar la funcionalidad existente. El objetivo es realizar un "paso a paso" que modifique cada elemento visual de la aplicación, desde la paleta de colores hasta la disposición de las pantallas, asegurando que cada cambio sea directamente aplicable sobre la estructura actual. Este plan se divide en tres fases lógicas: Consolidación y Preparación, Rediseño del Flujo Principal y Rediseño de Flujos Secundarios, culminando en una fase de validación final.

Fase 1: Consolidación y Preparación (Base de Datos y Tokens)

Esta fase inicial es crucial para crear una base sólida y consistente sobre la cual construir el nuevo diseño. No se modificará ninguna lógica de negocio, sino que se preparará la infraestructura de la UI.

- 1. Revisión y Migración Completa de Tokens Semánticos:** El primer paso es realizar un barrido exhaustivo de todos los archivos en el directorio `presentation/` (y subdirectorios) para identificar y reemplazar todos los colores codificados manualmente (ej. `Color(0xFF1B5E20)`) por sus equivalentes en `MaterialTheme.colorScheme` (ej. `MaterialTheme.colorScheme.primary`). Esto debe incluir colores de fondo, texto, tintes de botones, sombras y bordes. Se generará un informe de cambios que liste cada archivo modificado y la línea específica del cambio. Este paso asegura que toda la aplicación heredará automáticamente los colores de la paleta definida en el sistema de diseño Meridian.
- 2. Documentación de Reglas de Tipografía y Espaciado:** Antes de aplicar los nuevos estilos, se debe documentar explícitamente el sistema de tipografía y espaciado en un documento de estilo interno. Este documento especificará los tamaños y pesos de las fuentes para cada tipo de texto (Título, Cuerpo, Etiqueta, Botón) y definirá el sistema de espaciado basado en múltiplos de 8dp. Esta guía será la referencia obligatoria para todo el equipo durante las siguientes fases.
- 3. Auditoría de Componentes Base:** Se revisará el archivo `BocattaComponentsV2.kt` para asegurar que todos los componentes existentes ya utilicen los nuevos tokens semánticos, la tipografía definida y el sistema de espaciado. Si algún componente no cumple, se actualizará en esta fase. Por ejemplo, se verificará que todas las `BocattaButton` usen `padding(horizontal = 16.dp)` y `size(minimumHeight = 48.dp)`.

Fase 2: Rediseño del Flujo Principal (Ventas)

Con la base de tokens y componentes preparada, esta fase se enfoca en la pantalla principal de la aplicación, **SalesScreen**, que tiene el mayor impacto en la experiencia del usuario.

1. **Aplicación del Nuevo Diseño a SalesScreen:** Se procederá a rediseñar la pantalla de ventas completa. Esto implica:

- Aplicar la nueva tipografía y los tokens de color a todos los textos y fondos.
- Refactorizar el layout para que sea completamente adaptable. Se usará ``BoxWithConstraints`` para detectar el ancho del dispositivo y aplicar diferentes layouts:
- **Móvil ($<720dp$):** El catálogo de productos ocupará toda la pantalla. El panel del carrito se mostrará como un ``ModalBottomSheet`` deslizable desde la parte inferior, con una altura máxima del 50% de la pantalla. El botón de "COBRAR" se mantendrá fijo en la parte inferior cuando haya artículos en el carrito.
- **Tablet/POS ($\geq 720dp$):** Se usará un ``Row`` con dos columnas. La primera columna (catálogo) tendrá un peso de 0.62 y la segunda (carrito) un peso de 0.38. El panel del carrito será un ``Column`` fijo con un ancho mínimo de 320dp y máximo de 480dp, siempre visible a la derecha.
- Asegurar que la jerarquía visual priorice la información crítica: el total a pagar siempre será el elemento más grande y prominente en la pantalla.

1. **Optimización del Grid de Productos:** El `LazyVerticalGrid` que muestra los productos se actualizará para usar `GridCells.Adaptive(minSize = 160.dp)`, garantizando que el diseño se adapte a diferentes densidades de pantalla sin dejar espacios vacíos. Se implementará un debouncing de 300ms en el `TextField` de búsqueda para evitar recomposiciones excesivas.

2. **Refactorización de CarritoPanelV2:** El panel del carrito, ya extraído como un componente, se actualizará para que sus filas (`BocattaCartItemRow`) usen la nueva tipografía y los tokens de color. Se añadirá retroalimentación táctil a los botones de incrementar/decrementar cantidad.

Fase 3: Rediseño de Flujos Secundarios y Pantallas de Configuración

Una vez que el flujo principal esté completado y validado, se extenderá el nuevo sistema de diseño a todas las demás pantallas de la aplicación.

1. **Priorización y Rediseño de Pantallas Clave:** Se comenzará con las pantallas de mayor frecuencia de uso, como `InventarioScreen`, `ClientesScreen` y `ReportesScreen`. Cada una se refactorizará para usar los tokens semánticos, la

tipografía definida y los componentes `BocattaComponentsV2` (como `MetricCard` y `StatusBadge`).

2. **Implementación de Retroalimentación Visual Consistente:** Se añadirán animaciones sutiles, estados de carga en todos los botones que inician operaciones asíncronas y mensajes de error claros y no intrusivos (`Snackbar`) en todas partes. Por ejemplo, el botón de cobro mostrará un indicador de progreso mientras `SalesFlowUseCase` procesa la transacción.
3. **Actualización de Pantallas de Administración:** Pantallas como `AdminScreen` y `Configuración > Logística > Recetas` se actualizarán para usar el `DynamicFormEngine` con los nuevos componentes de formulario, asegurando que la configuración del sistema tenga la misma calidad visual que el flujo de ventas.

Este plan de acción asegura una transición gradual y controlada. Al dividir el trabajo en fases lógicas y centrarse primero en la base (tokens) y luego en los flujos individuales, se minimiza el riesgo de errores y se garantiza que el resultado final sea una experiencia de usuario homogénea, profesional y fiel a la identidad visual de Bocatta.

Implementación de Componentes Visuales Reutilizables y Adaptativos

La creación de una interfaz de usuario coherente, escalable y adaptable a distintos dispositivos es uno de los objetivos centrales de la investigación. Esto se logra mediante la implementación sistemática de componentes visuales reutilizables y un diseño responsive. La estrategia se basa en la extracción de bloques de construcción de la pantalla de ventas y su reutilización en todo el sistema, guiada por un breakpoint de diseño claro para diferenciar entre dispositivos móviles y tabletas.

El primer paso clave en este proceso fue la **extracción de componentes en `BocattaComponentsV2.kt`**. En lugar de mantener código duplicado dentro de `SalesScreen.kt`, se crearon funciones de composición reutilizables que encapsulan la lógica y el estilo de elementos comunes. Por ejemplo, el componente `ProductCardV2` fue diseñado para aceptar un `ProductoV2` y renderizar toda la tarjeta de producto, incluyendo el nombre, el precio y las etiquetas de stock. Esto no solo reduce la complejidad de `SalesScreen`, sino que también crea un único punto de mantenimiento para el diseño de los productos en toda la aplicación. De manera similar, el `CarritoPanelV2` fue extraído como un componente separado que recibe el estado del carrito como parámetros, desacoplando su presentación de la lógica del `ViewModel`.

La **implementación de un diseño adaptable** se logra principalmente a través de la función `BoxWithConstraints` en Jetpack Compose. Esta función permite al código adaptar su disposición basándose en el espacio de diseño disponible. El punto de ruptura seleccionado es de 720dp, que es un estándar común para distinguir entre teléfonos y tabletas [9](#) [75](#).

El diseño se adapta de la siguiente manera: - **En dispositivos móviles (ancho < 720dp):**
- El catálogo de productos ocupa todo el ancho disponible, organizado en un `LazyVerticalGrid`. - El panel del carrito se presenta como un `ModalBottomSheet`. Este diseño optimiza el espacio en una pantalla pequeña, mostrando el carrito de forma contextual y solo cuando el usuario lo solicita explícitamente. - El botón de "COBRAR", siendo una acción crítica, permanece fijo en la parte inferior de la pantalla para un acceso rápido, independientemente de la posición vertical de la pantalla. - **En tabletas y terminales POS (ancho ≥ 720dp):** - Se utiliza un `Row` para dividir la pantalla en dos columnas fijas. La primera columna contiene el catálogo de productos, y la segunda, el panel del carrito. - El panel del carrito se vuelve un componente fijo, siempre visible, con un ancho definido entre 320dp y 480dp. Esto permite al usuario ver los artículos seleccionados y el total en tiempo real mientras navega por el catálogo, mejorando la eficiencia operativa.

La siguiente tabla ilustra la diferencia en el diseño de la pantalla de ventas entre los dos puntos de ruptura:

Característica	Diseño para Teléfono (breakpoint < 720dp)	Diseño para Tableta/POS (breakpoint ≥ 720dp)
Layout Principal	Column con weight 1 para el contenido y weight 0 para el BottomBar.	Row con dos columnas (weight 0.62 para el catálogo, weight 0.38 para el carrito).
Panel del Carrito	ModalBottomSheet deslizable desde abajo, con altura máxima del 50%.	Column fija a la derecha, con ancho min/max definido (ej. 320dp a 480dp).
Acceso al Carrito	Accionado por un botón de "Ver carrito" en el BottomBar.	Siempre visible en la interfaz.
Botón de Cobro	Fijo en la parte inferior de la pantalla, fuera del scroll.	Dentro del panel del carrito.
Jerarquía Visual	Prioriza el catálogo de productos como el contenido principal.	Ofrece una vista panorámica de ambos, catalogo y carrito.

La implementación de estos diseños se realiza mediante la combinación de `BoxWithConstraints`, `Modifier.weight()` y `Modifier.fillMaxWidth()`. Por ejemplo, en el diseño de la tableta, la columna del catálogo se aplica `Modifier.weight(0.62f)` y `Modifier.fillMaxHeight()`, mientras que el panel del carrito recibe `Modifier.weight(0.38f)` y `Modifier.requiredWidthIn(min`

= 320.dp, max = 480.dp). Este enfoque declarativo y basado en compositores permite una implementación limpia y mantenible del diseño adaptable.

Finalmente, la consistencia visual se refuerza mediante el uso de los **componentes visuales reutilizables**. El `ProductCardV2`, ahora actualizado, se utiliza en el `LazyVerticalGrid` del catálogo. El `CarritoPanelV2` se compone de múltiples `BocattaCartItemRow`, cada uno representando un ítem del carrito. Este enfoque de "componentes en componentes" asegura que cualquier cambio en el diseño de una fila del carrito o de una tarjeta de producto se refleje instantáneamente en todas las instancias de esa componente en la aplicación, sin necesidad de buscar y reemplazar código en múltiples lugares. Este es el núcleo de un sistema de diseño efectivo: crear bloques de construcción tan robustos y flexibles que la creación de nuevas pantallas o la mejora de las existentes se vuelva un ejercicio de ensamblaje, no de escritura desde cero.

Estrategia de Validación, Accesibilidad y Optimización de Rendimiento

Una vez implementado el nuevo sistema de diseño, es imperativo ejecutar una estrategia de validación rigurosa para asegurar que la nueva interfaz de usuario no solo sea visualmente atractiva, sino también funcional, inclusiva y de alto rendimiento. Esta estrategia se centra en tres áreas críticas: el cumplimiento de los estándares de accesibilidad web (WCAG), la optimización del rendimiento de la interfaz de usuario y la validación exhaustiva a través de diversas pruebas.

La **accesibilidad** es un pilar fundamental para una aplicación profesional. El objetivo es cumplir con los niveles A y AA de las Web Content Accessibility Guidelines (WCAG) 2.2, el estándar internacional reconocido [41](#) [42](#). La estrategia de validación incluye: - **Contraste de Color:** Utilizar el escáner de accesibilidad de Android Studio o herramientas como aXe para verificar que todo el contenido textual y los gráficos de información tienen una relación de contraste mínima de 4.5:1 contra su fondo cercano [45](#). Los textos grandes (18pt o 14pt bold) tienen un requisito menor de 3:1 [40](#). Cualquier violación debe ser corregida ajustando los tokens de color en `MaterialTheme`. - **Contenido Alternativo:** Asegurar que todos los elementos no textuales (íconos, imágenes) que conveyen información tengan un atributo `contentDescription` adecuado. Los elementos puramente decorativos deben tener `contentDescription = ""` para ser ignorados por lectores de pantalla [40](#). - **Tamaño de Objetos de Entrada:**

Verificar que todos los elementos interactivos (botones, enlaces, entradas de formulario) tengan un tamaño de toque mínimo de 44x44 píxeles CSS (aproximadamente 48dp en Android), siguiendo el criterio de éxito WCAG 2.5.8 (Tamaño Mínimo del Objetivo) [23](#) [50](#). - **Navegación con Teclado:** Simular la navegación a través de la aplicación usando solo el D-pad o el trackball para asegurar que el indicador de foco sea visible en todo momento y que no haya "trampas de teclado" que atrapen al usuario [40](#). - **Pruebas con TalkBack:** Realizar pruebas de usabilidad con el lector de pantalla de Android habilitado para confirmar que la narración de la interfaz es lógica, completa y útil para usuarios con discapacidad visual [44](#).

La **optimización del rendimiento de la interfaz de usuario** es crucial para mantener una experiencia fluida, especialmente en dispositivos de gama baja. Las medidas a tomar incluyen: - **Evitar Recomposiciones Ineficientes:** Utilizar `derivedStateOf` para calcular valores costosos dentro de los composites y asegurar que los modificadores de lista (`LazyColumn`, `LazyVerticalGrid`) usen claves únicas para sus elementos (`items(key = { it.id }) { ... }`). Esto evita que la IU se reconstruya innecesariamente cuando solo cambia un pequeño fragmento de datos [114](#). -

Optimización de Imágenes y Animaciones: Asegurar que todas las imágenes sean de formato WebP y estén optimizadas. Limitar el uso de animaciones complejas y costosas en recursos, reservándolas para casos de uso importantes como la retroalimentación de la acción del usuario. - **Medición de Latencia:** Utilizar la herramienta de "Profiler" de Android Studio para medir el tiempo de composición de los fotogramas (frame composition time). El objetivo es mantenerlo por debajo de 16ms para lograr 60fps. Identificar y optimizar cualquier tarea pesada que se ejecute en el hilo principal.

La **validación exhaustiva** se llevará a cabo a través de una batería de pruebas: - **Pruebas Unitarias:** Ya existen 218 tests unitarios que cubren la lógica de dominio. Se deben agregar pruebas para las nuevas funciones de UI. - **Pruebas Instrumentadas (UI Tests):** Es fundamental añadir un conjunto de pruebas instrumentadas utilizando `ComposeTestRule`. Estas pruebas automatizan la interacción con la IU para validar flujos críticos, como: - Agregar un producto al carrito y verificar que el total se actualice correctamente. - Activar/desactivar el modo avión y validar que el indicador de red y el estado del botón de cobro cambien como se espera. - Validar el cambio de diseño en el breakpoint de 720dp [28](#). - **Pruebas de Integración:** Utilizar el emulador de Firestore para simular diferentes condiciones de red (conexión lenta, intermitente, desconectado) y validar que el flujo de venta offline funciona correctamente, que la cola de sincronización se gestiona bien y que la aplicación se sincroniza correctamente una vez que se recupera la conexión. - **Pruebas de Campo:** Antes del lanzamiento, es crucial desplegar una versión de la aplicación con el nuevo diseño en dispositivos físicos (teléfonos y tabletas)

en un entorno de prueba controlado. Recopilar feedback directamente de los usuarios finales sobre la usabilidad, la velocidad y la claridad de la nueva interfaz.

Al ejecutar esta estrategia multifacética, se puede tener confianza en que el rediseño de la interfaz de usuario no solo cumple con los requisitos estéticos, sino que también proporciona una experiencia robusta, inclusiva y de alto rendimiento que justifica la transición de un "esqueleto" funcional a una herramienta profesional de clase mundial.

Hoja de Ruta para la Transición a una Experiencia de Usuario Profesional

La transición de Bocatta POS de una aplicación funcional a una herramienta visualmente potente y profesional es un proceso estratégico que culmina en la fase actual de rediseño de la interfaz de usuario. La hoja de ruta propuesta no busca una simple actualización estética, sino una transformación sistémica que alinee la experiencia del usuario con la solidez arquitectónica que se ha construido previamente. Este documento final sirve como el plan maestro para completar esta transición, asegurando que el resultado sea una aplicación de producción, intuitiva, eficiente y fiel a la identidad de la marca.

La trayectoria del proyecto ha sido notable. Se ha pasado de una arquitectura monolítica acoplada a un **Modular Monolith** que respeta plenamente los principios de Clean Architecture [119](#). La inversión de dependencias, el dominio puro y la separación de lógicas financieras y físicas no son meras abstracciones, sino la infraestructura que hace posible la flexibilidad y la calidad visual solicitadas. La existencia de un motor de formularios dinámicos (DynamicFormEngine) es particularmente significativa; significa que la capacidad de adaptar la interfaz a diferentes giros comerciales ya está incrustada en la lógica del sistema, listo para ser activada mediante la configuración de datos en Firestore [96 107](#). El rediseño de la interfaz de usuario, por lo tanto, no requiere volver a conectar la lógica, sino simplemente proveerle un nuevo "vestido" consistente y moderno.

La hoja de ruta final se articula en torno a la ejecución del plan de acción detallado en las secciones anteriores. El primer paso, la **consolidación de tokens semánticos y la documentación de reglas de diseño**, establece la base de oro para toda la aplicación. Al erradicar los colores duros y definir claramente la tipografía y el espaciado, se crea un sistema de diseño coherente que guiará todas las decisiones de diseño futuras. La segunda fase, el **rediseño del flujo principal de ventas**, es el corazón del proyecto. Al aplicar el nuevo sistema de diseño a SalesScreen y sus componentes

(ProductCardV2, CarritoPanelV2), y al implementar el diseño adaptable para móviles y tabletas, se transforma la experiencia central del usuario. La tercera fase, el **rediseño de los flujos secundarios y pantallas de configuración**, asegura que esta calidad visual se extienda a todas las áreas de la aplicación, eliminando inconsistencias y elevando el nivel general de la interfaz. Finalmente, la **estrategia de validación** —que abarca accesibilidad, rendimiento y pruebas exhaustivas— garantiza que el producto resultante no solo se vea bien, sino que sea robusto, inclusivo y rápido.

El cumplimiento de esta hoja de ruta resultará en un producto que satisface plenamente el objetivo de investigación original. La aplicación dejará de sentirse como un "esqueleto" para convertirse en una herramienta visualmente potente y profesional. La eficiencia operativa mejorará gracias a una jerarquía visual clara y un diseño optimizado para el punto de venta. La legibilidad a distancia y la accesibilidad se verán garantizadas, protegiendo a la empresa de posibles problemas y ampliando su base de usuarios potenciales. La consistencia a través de todos los dispositivos y flujos de trabajo creará una experiencia de usuario fluida y predecible.

En conclusión, Bocatta POS ha alcanzado un punto de inflexión. Ha construido una arquitectura de software excepcionalmente sólida. Ahora, con este informe como guía, el equipo está equipado para ejecutar el plan de acción definitivo que vestirá esta arquitectura con una interfaz de usuario moderna, coherente y profesional. La transición está completa, y el resultado será una plataforma de punto de venta verdaderamente de nivel empresarial.

Referencia

1. Using ecosystem services-based approaches in Integrated Water ... <https://unesdoc.unesco.org/ark:/48223/pf0000368517>
2. Chan Kah Kiong - Executive Leader in Data Analytics & Business ... <https://sg.linkedin.com/in/chan-kah-kiong-b70b5a3>
3. Assessment of User Experience (UX) Design Trends in Mobile ... https://www.researchgate.net/publication/382804453_Assessment_of_User_Experience_UX_Design_Trends_in_Mobile_Applications

4. 7 Mobile UX/UI Design Patterns Dominating 2026 - Sanjay Dey <https://www.sanjaydey.com/mobile-ux-ui-design-patterns-2026-data-backed/>
5. Guidance on Applying WCAG 2 to Non-Web Information and ... - W3C <https://www.w3.org/TR/wcag2ict-22/>
6. Fitts' Law: Designing Touch Targets for Accuracy | Siddhant ... https://www.linkedin.com/posts/siddhantshandilya_fittss-law-the-time-to-acquire-a-target-activity-7418946451537375232-Hr8g
7. 101 Digital Accessibility (a11y) tips and tricks - DEV Community <https://dev.to/grahamthedev/101-digital-accessibility-tips-and-tricks-4728>
8. 17 POS UI ideas | pos, pos design, point of sale - Pinterest <https://www.pinterest.com/salimmohsen/pos-ui/>
9. POS System Design Principles to Know (2024) - Shopify <https://www.shopify.com/blog/pos-system-design>
10. POS UI design & development: what should be included & avoided? <https://stackoverflow.com/questions/1108799/pos-ui-design-development-what-should-be-included-avoided>
11. Best Point of Sale Software in India (2026) - Techimply <https://www.techimply.com/software/point-of-sale-software>
12. glove.6B.100d.txt-vocab.txt - CodaLab Worksheets <https://worksheets.codalab.org/rest/bundles/0xadf98bb30a99476ab56ebff3e462d4fa/contents/blob/glove.6B.100d.txt-vocab.txt>
13. BLAST ANNUAL 2020 PDF VOL 1 FINAL VERSION SEPT ... - Scribd <https://www.scribd.com/document/538547158/BLAST-ANNUAL-2020-PDF-VOL-1-FINAL-VERSION-SEPT-26-2020-mfey9q>
14. The VentureFizz Podcast - TopPodcast.com https://toppodcast.com/podcast_feeds/the-venturefizz-podcast/
15. Normal 60b3e4c68e855 | PDF | World Wide Web - Scribd <https://www.scribd.com/document/537503706/normal-60b3e4c68e855>
16. Web-Based POS System for Coffee Shalter | PDF | Point Of Sale <https://www.scribd.com/document/906666043/DEVELOPMENT-OF-A-WEB-BASED-POINT-OF-SALES-POS-INFORMATION-SYSTEM-FOR-COFFEE-SHALTER>
17. (PDF) Design and Development of a Web-Based Point of Sale ... https://www.researchgate.net/publication/394571608_Design_and_Development_of_a_Web-Based_Point_of_Sale_System_for_Small-Scale_Retail_Management
18. Master Your Restaurant Operations 9 Essential POS Features You ... <https://clotouch.com/blog/master-restaurant-operations-9-essential-pos-features-bbb/>
19. Data Collection Methods: Tools for Better Business Insights - Artsyl <https://www.artsyltech.com/blog/data-collection-methods>

20. MERIDIAN中文(繁体)翻译: 剑桥词典 <https://dictionary.cambridge.org/zhs/%E8%AF%8D%E5%85%B8/%E8%8B%B1%E8%AF%AD-%E6%B1%89%E8%AF%AD-%E7%B9%81%E4%BD%93/meridian>
21. MERIDIAN Definition & Meaning - Dictionary.com <https://www.dictionary.com/browse/meridian>
22. words-333333 - cs.Princeton <https://www.cs.princeton.edu/courses/archive/spring18/cos226/assignments/autocomplete/testing/words-333333.txt>
23. Understanding Success Criterion 2.5.8: Target Size (Minimum) - W3C <https://www.w3.org/WAI/WCAG22/Understanding/target-size-minimum.html>
24. Technology for inclusion - UNESCO Digital Library <https://unesdoc.unesco.org/ark:/48223/pf0000373655>
25. XBRL Viewer - SEC.gov <https://www.sec.gov/ix?doc=/Archives/edgar/data/1540305/000089418921008279/ck0001540305-20201231.htm>
26. [PDF] Improving ICT Integration in Today's AEC Industry - CORE <https://core.ac.uk/download/pdf/10878335.pdf>
27. TouchBistro vs Lightspeed vs Toast vs Square vs Foodship - LinkedIn <https://www.linkedin.com/pulse/touchbistro-vs-lightspeed-toast-square-foodship-r-m-mtrqf>
28. Top 5 iOS Mobile App Accessibility Issues Pt 1. - DEV Community <https://dev.to/steady5063/top-5-ios-mobile-app-accessibility-issues-pt-1-51g8>
29. Web Content Accessibility Guidelines (WCAG) 2.2 - W3C <https://www.w3.org/TR/WCAG22/>
30. Designing for Touch: Best Practices for Mobile UX/UI - LinkedIn <https://www.linkedin.com/pulse/designing-touch-best-practices-mobile-uxui-james-griffin-hxwje>
31. Touch-Friendly UI Design: Best Practices to Ensure Seamless ... https://dev.to/okoye_ndidiamaka_5e3b7d30/touch-friendly-ui-design-best-practices-to-ensure-seamless-mobile-interactions-4b1c
32. Meridian Systems POS Strategy Case Study - Swot Analysis - Scribd <https://www.scribd.com/presentation/473737632/Meridian-Systems-copy-pptx>
33. [PDF] IBM Software Case Studies <https://public.dhe.ibm.com/software/smb/casestudies/expresscasestudy1.pdf>
34. [PDF] The Eyes Have It - AMiner https://static.aminer.org/pdf/PDF/000/254/953/multipar_an_output_queue_atm_modular_switch_with_multiple_phases.pdf
35. Accessibility evaluation of interaction modalities and cognitive ... <https://www.sciencedirect.com/science/article/pii/S095058492500309X>

36. (PDF) The performance of touch screen soft buttons - ResearchGate https://www.researchgate.net/publication/221514597_The_performance_of_touch_screen_soft_buttons
37. Unpacking Customer Experience in Online Shopping: Effects ... - MDPI <https://www.mdpi.com/0718-1876/20/3/245>
38. Enhancing supermarket robot interaction: an equitable multi-level ... <https://www.frontiersin.org/journals/robotics-and-ai/articles/10.3389/frobt.2025.1576348/full>
39. Adaptique: Multi-objective and Context-aware Online Adaptation of ... <https://arxiv.org/html/2508.08505v1>
40. IBM accessibility requirements <https://www.ibm.com/able/requirements/requirements/>
41. WCAG 2 Overview | Web Accessibility Initiative (WAI) - W3C <https://www.w3.org/WAI/standards-guidelines/wcag/>
42. WCAG 2.2 is an ISO Standard – What it means for accessibility <https://whoisandywhite.com/2025/11/wcag-2-2-becomes-an-iso-standard-what-it-means-for-web-designers-and-developers/>
43. (PDF) How to design font size for older adults: A systematic literature ... https://www.researchgate.net/publication/362402229_How_to_design_font_size_for_older_adults_A_systematic_literature_review_with_a_mobile_device
44. A Systematic Review of Accessible Payment Solutions for All https://www.researchgate.net/publication/391585777_Empowering_Visually_Impaired_Consumers_in_Retail_A_Systematic_Review_of_Accessible_Payment_Solutions_for_All
45. Improve Design Accessibility - Trailhead <https://trailhead.salesforce.com/content/learn/modules/designing-for-web-accessibility/integrate-accessibility-into-your-design>
46. Mastercard Branding Requirements <https://www.mastercard.com/brandcenter/us/en/brand-requirements/mastercard.html>
47. Meridian Systems POS Strategy Case Study - Swot Analysis - Scribd <https://fr.scribd.com/presentation/473737632/Meridian-Systems-copy-pptx>
48. Publications - Tableau <https://www.tableau.com/research/publications>
49. Toward Unified Guidelines for Kiosk Accessibility - ResearchGate https://www.researchgate.net/publication/334073148_Toward_Unified_Guidelines_for_Kiosk_Accessibility
50. Understanding Success Criterion 2.5.5: Target Size | WAI - W3C <https://www.w3.org/WAI/WCAG21/Understanding/target-size.html>

51. Restaurant Cash Register - Android POS System with Touch Screen <https://www.alibaba.com/showroom/restaurant-cash-register.html>
52. UAAG 2.0 Reference: Explanations, Examples, and Resources for ... <https://www.w3.org/TR/UAAG20-Reference/>
53. [PDF] Improving Content Design on Mobile Devices to Reduce Situational ... https://www.researchgate.net/profile/Garreth_Tigwell/publication/334290097_Improving_Content_Design_on_Mobile_Devices_to_Reduce_Situational_Visual_Impairments/links/5d23006e92851cf4406f4fcd/Improving-Content-Design-on-Mobile-Devices-to-Reduce-Situational-Visual-Impairments.pdf
54. Revolutionizing Workflows: The Role of Efficiency in Organizational ... https://www.researchgate.net/publication/384298740_Revolutionizing_Workflows_The_Role_of_Efficiency_in_Organizational_Success
55. (PDF) Improving order-picking process through implementation of ... https://www.researchgate.net/publication/328112783_Improving_order-picking_process_through_implementation_of_warehouse_management_system
56. Digital Technologies in Hospital Pharmacy: A Systematic Review of ... <https://pmc.ncbi.nlm.nih.gov/articles/PMC12883916/>
57. Enhancing procedure quality: Advanced language tools for ... <https://www.sciencedirect.com/science/article/pii/S0951832025005095>
58. The impact of cloud computing technology on cloud accounting ... <https://www.nature.com/articles/s41599-025-05190-3>
59. (PDF) Evaluating the role of ERP systems in streamlining order ... https://www.researchgate.net/publication/390678834_Evaluating_the_role_of_ERP_systems_in_streamlining_order_management_and_inventory_optimization_for_US_retailers
60. Blockchain-Enabled Supply Chain Management: A Review ... - MDPI <https://www.mdpi.com/2076-3417/15/9/5168>
61. Blockchain in supply chain management: a comprehensive review ... <https://link.springer.com/article/10.1007/s11301-025-00546-0>
62. AI-enhanced Digital Twin systems for warehouse logistics optimization <https://www.sciencedirect.com/science/article/pii/S2405959526000093>
63. Capstone project - Kaggle <https://www.kaggle.com/code/gauravsrivastav2507/capstone-project>
64. Android-Based Emergency Medical Service in Ethiopian Health ... <https://www.scribd.com/document/489535916/Android-based-Emergency-Medical-service-in-Ethiopian-Health-care-Institutions-Alazar-mengstab-pdf>

65. BCTG | Interviewing - Bull City Talent Group <https://bullcitytalent.com/tag/interviewing/>
66. What is Meridian Audio Technology? | LG UK https://www.lg.com/uk/lg-experience/lg-lab/who-are-meridian-audio/?srsltid=AfmBOoqjgw251p6qiqwD_LKekOjjMhH4Y8V21jkotsYMdVY-qhluALoL
67. G1000 NXi Upgrade Certified in Piper Meridian | Garmin Blog <https://www.garmin.com/en-US/blog/aviation/g1000-nxi-upgrade-certified-in-piper-meridian/>
68. [PDF] Mapping of international Internet public policy - UNCTAD https://unctad.org/system/files/official-document/ecn162015crp2add1_en.pdf
69. [PDF] ADMINISTRATOR'S GUIDE - Meridian https://help.meridian360.com/2022/PDFs/Meridian2022_AG_LTR.pdf
70. [PDF] ADMINISTRATOR'S GUIDE - Meridian https://help.meridian360.com/2021_R2/PDFs/Meridian2021_R2_AG_LTR.pdf
71. [PDF] Coast to Coast Path National Trail - Accessibility Audit - AWS <https://nationaltrails.s3.eu-west-2.amazonaws.com/uploads%2FNE-Coast-to-Coast-Path-National-Trail-Access-Report-September-2024-FINAL.pdf>
72. Understanding Success Criterion 2.5.5: Target Size (Enhanced) | WAI <https://www.w3.org/WAI/WCAG22/Understanding/target-size-enhanced.html>
73. EARTH OBSERVATION PLATFORM FOR SEISMIC MONITORING ... https://www.academia.edu/165576670/EARTH_OBSERVATION_PLATFORM_FOR_SEISMIC_MONITORING_A_Technical_Guide_for_Space_Data_Integration
74. [PDF] Project Meridian: Simplifying transactions through innovation <https://www.bis.org/publ/othp63.pdf>
75. [PDF] Zebra Point of Sale (POS) Solution Brief https://www.zebra.com/content/dam/zebra_dam/en/brief/solution/zebra-point-of-sale-solution-brief-solution-en-us.pdf
76. (PDF) Mobile point of sale design and implementation - ResearchGate https://www.researchgate.net/publication/327901030_Mobile_point_of_sale_design_and_implementation
77. POS - Mobile POS - USI https://www.usiglobal.com/en/products?c_id=Mobile-POS
78. The Best POS Systems for 2026 - Business News Daily <https://www.businessnewsdaily.com/2955-best-pos-systems.html>
79. Best Restaurant Management System Online POS | POSQuote <https://posquote.com/best-restaurant-management-system-online-pos-posquote/>
80. hw3_stats_google_1gram.txt - CMU School of Computer Science https://www.cs.cmu.edu/~roni/11661/2017_fall_assignments/hw3_stats_google_1gram.txt

81. 5 Best iPad Restaurant POS Systems | Compare Top Software <https://posquote.com/ipad-restaurant-pos-systems/>
82. Android Touch Target Size Guidelines - Accessibility - Scribd <https://www.scribd.com/document/859934395/Touch-Target-Size-Android-Accessibility-Help>
83. [PDF] ADMINISTRATOR'S GUIDE - Meridian https://help.meridian360.com/2024/PDFs/Meridian2024_AG_LTR.pdf
84. [PDF] SICAM Multifunctional Recorder SICAM Q200 7KG97 https://cache.industry.siemens.com/dl/files/896/109744896/att_931852/v1/SICAM_Q200_7KG97_MAN_US.pdf
85. Advances in Intelligent and Soft Computing 142 - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-3-642-27957-7.pdf>
86. [PDF] MAY 6, 1998 - | WA.gov <https://lawfilesexternal.wa.gov/law/wsr/1998/09/98-09.pdf>
87. [PDF] STM ship systems ready for installation - AWS https://stm-stmvalidation.s3.eu-west-1.amazonaws.com/uploads/20200225090155/STMVal_D2.5-STM-ship-systems-ready-for-installation-1.pdf
88. Toast vs Square: 2024 POS Comparison | PDF | Point Of Sale - Scribd <https://www.scribd.com/document/747920560/Toast-Vs-Square-2024-Comparison>
89. Pos System Ui Design - Pinterest <https://www.pinterest.com/ideas/pos-system-ui-design/927556693105/>
90. Tailwind's Impact on HTML Structure and Code Complexity - LinkedIn https://www.linkedin.com/posts/markshust_so-many-developers-disagree-with-this-post-activity-7424265579680858112-Tqfk
91. Flowbite - Build websites even faster with components on top of ... <https://flowbite.com/>
92. Modernize POS journeys with React and Fluent UI - Microsoft Learn <https://learn.microsoft.com/en-us/dynamics365/release-plan/2026wave1/enterprise-resource-planning/dynamics365-commerce/modernize-pos-journeys-react-fluent-ui>
93. Atlassian Frontend Development <https://atlaskit.atlassian.com/>
94. Free Design system templates collection | Confluence - Atlassian <https://www.atlassian.com/software/confluence/templates/collections/designs-at-scale>
95. Example - Theming components - Atlaskit - Atlassian <https://atlaskit.atlassian.com/examples/core/theme/theming-components>
96. Design system template | Confluence - Atlassian <https://www.atlassian.com/software/confluence/templates/design-system>
97. How HTML Powers the Frontend of Point of Sale (POS) Systems <https://dev.to/kararimartin/how-html-powers-the-frontend-of-point-of-sale-pos-systems-f59>

98. How to avoid business logic in frontend while ... - Stack Overflow <https://stackoverflow.com/questions/65898705/how-to-avoid-business-logic-in-frontend-while-keeping-frontend-interactive>
99. UX Design for Retail POS & Inventory Management | Case Study <https://www.rossul.com/portfolio/retail-pos-system-ux/>
100. Discover 45 POS UI and pos interface design ideas - Pinterest <https://www.pinterest.com/ultracao/pos-ui/>
101. 333333 23135851162 the 13151942776 of 12997637966 <ftp://ftp.cs.princeton.edu/pub/cs226/autocomplete/words-333333.txt>
102. [PDF] Insight MFR Names - State of Iowa Bid Opportunities <https://bidopportunities.iowa.gov/Home/ShowContractDocument/233de3c6-b20e-480c-a6f0-92e3176d4eb2>
103. Change app theme during runtime in Angular - Stack Overflow <https://stackoverflow.com/questions/57967054/change-app-theme-during-runtime-in-angular>
104. Building a Point of Sale (POS) Software Frontend with Angular <https://www.linkedin.com/pulse/building-point-sale-pos-software-frontend-angular-kushagra-infotech-69vfc>
105. getting started – AWS Startups Blog <https://aws.amazon.com/blogs/startups/tag/getting-started/feed/>
106. AUI, design tokens and original theme - how to handle it <https://community.developer.atlassian.com/t/aui-design-tokens-and-original-theme-how-to-handle-it/71583>
107. Theming Connect Apps - Developer, Atlassian <https://developer.atlassian.com/cloud/confluence/connect-theming/>
108. Introducing design tokens, new colour foundations and dark mode <https://community.developer.atlassian.com/t/introducing-design-tokens-new-colour-foundations-and-dark-mode/62258>
109. Design tokens and theming - Atlassian Developer <https://developer.atlassian.com/platform/forge/design-tokens-and-theming/>
110. Our new home for Atlassian Design System <https://community.developer.atlassian.com/t/our-new-home-for-atlassian-design-system/40698>
111. Upcoming changes in design tokens 1.0.0 — action required soon ... <https://community.developer.atlassian.com/t/upcoming-changes-in-design-tokens-1-0-0-action-required-soon-edit-now-live/66243>
112. Changelog - Tokens - Atlassian Frontend Development - Atlaskit <https://atlas.kit.atlassian.com/packages/design-system/tokens/changelog>

113. A visual refresh of our UI foundations is coming <https://community.developer.atlassian.com/t/a-visual-refresh-of-our-ui-foundations-is-coming/84949>
114. Frontend System Design: Scalable CSS Architecture <https://dev.to/zeeshanali0704/frontend-system-design-scalable-css-architecture-472n>
115. Theming overview - Commerce | Dynamics 365 - Microsoft Learn <https://learn.microsoft.com/en-us/dynamics365/commerce/e-commerce-extensibility/theming>
116. [PDF] Implementation Guide - Success by Design <https://download.microsoft.com/download/c/2/4/c24f97f7-00a6-46db-9b50-8ff6e03e9d45/Dynamics%20365%20Implementation%20Guide%20v1-2.pdf>
117. Tech Blog | Mobile App, eCommerce, Salesforce Insights - Emizentech <https://emizentech.com/blog/>
118. Legacy Software Modernization - Frontend Assessment - TeemBost <https://teembost.com/w/legacy-software-modernization-data-foundation>
119. Strategies for Modernizing Legacy Systems - LinkedIn <https://www.linkedin.com/pulse/strategies-modernizing-legacy-systems-sundar-govindarajan-nbime>
120. Metadata and Semantic Research - Springer Nature <https://link.springer.com/content/pdf/10.1007/978-3-031-81974-2.pdf>
121. Articles Home | New Era Technology <https://digital.neweratech.com/articles>